4

信号是 Unix/Linux 系 中 程 通信和 步事件通知的基 机制。 程可以通 信号相互通知事件，内核也会在 常、 端中断或子 程状 化 向 程 送信号。StarryOS 在 acf07fd8 moss 在此基 上完善了停止与 、权限 、 器 截和父 程通知等 路径，使信号机制能 支持作 控制、ptrace POSIX 信号 。

本章先 明信号的基本概念和 StarryOS 已有的基 ，再分析 moss

4.1

4.1.1

信号是一 量 的 程 通信机制，用于通知 程 生了某个事件。与管道、消息 列等需要 式数据 的机制不同，信号只携 事件类型和少量附加信息。 程收到信号后可以 行默 操作、忽略信号或 用用 注册的 理函数。

Linux 支持 64 1-31 32-64

每个信号都有默 行 ，分 五类：

- **终止**（Terminate）： SIGTERM、SIGINT
- **忽略**（Ignore）：不做任何 理，如 SIGCHLD、SIGWINCH
- **核心转储**（CoreDump）： SIGSEGV、SIGABRT
- **停止**（Stop）： SIGSTOP、SIGTSTP
- **继续**（Continue）：恢 已停止的 程，如 SIGCONT

sigaction 修改信号 理方式，但 SIGKILL 和 SIGSTOP 不能被捕 、阻塞或忽略，保 了系 始 能 止或停止任意 程。

4.1.2 StarryOS

StarryOS 在 acf07fd8

- **信号发送**：kill、tkill、tgkill 可以向 程或 程 送信号
- **信号接收**： 程和 程各自 待 理信号 列
- **信号处理**：用 信号 理函数的 造与恢
- **信号屏蔽**：sigprocmask 可以阻塞部分信号的
- **信号等待**：sigtimedwait、sigsuspend 主 等待信号到

底 starry-signal 提供了 程 和 程 的信号管理器。 程 管理器 ThreadSignalManager 保存每个 程的待 理信号、屏蔽集和信号 ； 程 管理器 ProcessSignalManager 保存 程共享的信号 列和信号 理 作表。 送到 程的信号会被分配 其中一个未屏蔽 信号的 程。

信号 生在系 用返回用 之前。内核 用 check_signals 遍 待 理信号，根据信号 理 作决定是 用用 理函数、 行默 操作 是 等待。用 理函数通 在用

上 造 SignalFrame

sigreturn 恢 。

4.1.3 StarryOS

4.2

4.2.1

信号 送从系 用 始。kill 向 程或 程 送信号，tkill 向 程 送信号，tgkill 向指定 程内的 程 送信号。用 程序也可以通 raise 向自己 送信号。

```
pub fn sys_kill(pid: i32, signo: u32) -> AxResult<isize> {
    let sig = make_siginfo(signo, SI_USER as _)?;
    let sig_signo = sig.as_ref().map(|sig| sig.signo());
    let curr = current();
    let caller = &curr.as_thread().proc_data;

    match pid {
        1.. => {
            // 向指定 程 送信号
            let target = get_process_data(pid as _)?;
            send_signal_to_process_checked(caller, &target, sig)?;
        }
        0 => {
            // 向当前 程 送信号
            let pgid = caller.proc.group().pgid();
            send_signal_to_process_group_checked(caller, pgid, sig)?;
        }
        -1 => {
            // 向所有有 限的 程 送信号
            // ...
        }
        ..-1 => {
            // 向指定 程 送信号
            let pgid = pid.checked_neg().ok_or(AxError::NoSuchProcess)? as Pid;
            send_signal_to_process_group_checked(caller, pgid, sig)?;
        }
    }
    Ok(0)
}
```

代 4-1 sys_kill 的信号 送

信号 送前先 造 SignalInfo，包含信号 号、 送原因代 和 送者 程 ID。signo 0

4.2.2

信号 送后 入接收方的待 理 列。moss 使用 列 ： 程 列和 程 列。 送到 程的信号 入 程 列， 送到 程的信号 入 程 列， 先 程 列再 程 列。

```rust
pub struct ThreadSignalManager {
    proc: Arc<ProcessSignalManager>,
    pending: SpinNoIrq<PendingSignals>,
    blocked: SpinNoIrq<SignalSet>,
    possibly_has_signal: AtomicBool,
    // ...
}

pub struct ProcessSignalManager {
    pending: SpinNoIrq<PendingSignals>,
    actions: Arc<SpinNoIrq<SignalActions>>,
    children: SpinNoIrq<Vec<(u32, Weak<ThreadSignalManager>)>>,
    possibly_has_signal: AtomicBool,
    // ...
}
```

代　4-2

PendingSignals 区分　准信号和　　信号。　准信号只保存一份，重　　送会被合并；信号使用　列保存，按　送　序排　。

```rust
pub struct PendingSignals {
    pub set: SignalSet,
    info_std: [Option<Box<SignalInfo>>; 32],
    info_rt: [VecDeque<SignalInfo>; 33],
}

pub fn put_signal(&mut self, sig: SignalInfo) -> bool {
    let signo = sig.signo();
    let added = self.set.add(signo);

    if signo.is_realtime() {
        self.info_rt[signo as usize - 32].push_back(sig);
    } else {
        if !added {
            //
            return false;
        }
        self.info_std[signo as usize] = Some(Box::new(sig));
    }
    true
}
```

代　4-3　待　理信号的排

信号入　后　置 possibly_has_signal

4.2.3

信号　　生在系　　用返回用　　之前。内核从待　理信号中　出一个未被屏蔽的信号，根据信号　理　作决定如何　理。

```rust
pub fn check_signals(
    thr: &Thread,
    uctx: &mut UserContext,
    restore_blocked: Option<SignalSet>,
) -> bool {
    // ptrace
    crate::syscall::ptrace_wait_if_stopped(thr);

    let pid = thr.proc_data.proc.pid();
    if crate::syscall::ptrace_is_traced(pid) {
        let mask = !thr.signal.blocked();
        if let Some(sig) = thr.signal.dequeue_signal(&mask) {
            let signo = sig.signo();
            if crate::syscall::ptrace_intercept_signal(thr, signo) {
                return true;
            }
            do_exit(signo as i32, true);
        }
    }

    let Some((sig, os_action)) = thr.signal.check_signals(uctx,
restore_blocked) else {
        return false;
    };

    let signo = sig.signo();
    match os_action {
        SignalOSAction::Terminate => {
            do_exit(signo as i32, true);
        }
        SignalOSAction::CoreDump => {
            do_exit((signo as i32) | 0x80, true);
        }
        SignalOSAction::Stop => {
            *thr.proc_data.wait_status.lock() =
Some(ProcessWaitStatus::Stopped(signo));
            wake_waiting_parent(&thr.proc_data);
            wait_for_continue(&thr.proc_data);
        }
        SignalOSAction::Continue => {
            *thr.proc_data.wait_status.lock() =
Some(ProcessWaitStatus::Continued(signo));
            wake_waiting_parent(&thr.proc_data);
        }
        SignalOSAction::Handler => {
            // 用    理函数已    置，直接返回用      行
        }
    }
    true
}
```

代 4-4 信号   与 理的主流程

内核首先 程是否正在被 ptrace

信号 理 作分 五 ：

- Terminate： 程退出，退出 信号 号
- CoreDump： 程退出并 置 core dump 1）
- Stop： 程 入停止状 ，等待 SIGCONT 或 SIGKILL 恢
- Continue：从停止状 恢 ， 程 行
- Handler： 用用 注册的信号 理函数

用 理函数通 修改用 寄存器 。内核在用 上 造 SignalFrame，保存被中断的上下文、信号信息和恢 用的 ucontext，再把指令指 指向用 理函数、参数寄存器指向 ，返回地址 sigreturn 系 用的入口。用 理函数 行完 后自 用 sigreturn，内核从 恢 寄存器状 ， 程 原来的 行流。

4.3

4.3.1

POSIX root 用 可以向任意 程 送信号。moss 在 send_signal_to_process_checked 中 了 一 ：

```
fn can_send_signal(caller: &ProcessData, target: &ProcessData, signo:
Option<Signo>) -> bool {
    caller.euid() == 0
        || caller.uid() == target.uid()
        || caller.uid() == target.suid()
        || caller.euid() == target.uid()
        || caller.euid() == target.suid()
        || (signo == Some(Signo::SIGCONT)
            && caller.proc.group().session().sid() ==
target.proc.group().session().sid())
}

fn check_send_signal_permission(
    caller: &ProcessData,
    target: &ProcessData,
    signo: Option<Signo>,
) -> AxResult<()> {
    if can_send_signal(caller, target, signo) {
        Ok(())
    } else {
        Err(AxError::from(LinuxError::EPERM))
    }
}
```

代 4-5 信号 送权限

权限 包括：

1. root 用　（`euid == 0`）可以向任意　程　送信号
2. 　　ID 与目　　程的　　或保存的用　ID 匹配　允　　送
3. `SIGCONT` 在同一会　内可以跨用　　送，用于作　控制

　　`kill`、`tkill`、`tgkill` 和 `rt_sigqueueinfo` 等所有信号　送系　　用。

4.3.2

`kill` 的 `pid` 参数　0　　−1

moss 在　些　景下也　行权限　　：

```
fn send_signal_to_process_group_checked(
    caller: &ProcessData,
    pgid: Pid,
    sig: Option<SignalInfo>,
) -> AxResult<()> {
    let pg = get_process_group(pgid)?;
    let signo = sig.as_ref().map(|sig| sig.signo());
    let mut found = false;
    let mut permitted = false;

    for proc in pg.processes() {
        let Ok(target) = get_process_data(proc.pid()) else {
            continue;
        };
        found = true;

        if can_send_signal(caller, &target, signo) {
            permitted = true;
            if let Some(sig) = sig.as_ref() {
                let _ = send_signal_to_process(target.proc.pid(),
Some(sig.clone()));
            }
        }
    }

    if permitted {
        Ok(())
    } else if found {
        Err(AxError::from(LinuxError::EPERM))
    } else {
        Err(AxError::NoSuchProcess)
    }
}
```

代　4-6

`EPERM`；　程　不存在或　空　返回 `ESRCH`。

广播 送（pid == -1）遍 系 中所有 程，跳 init

killall 等批量信号 送工具。

4.3.3

moss tkill 和 tgkill 要求 程 ID 和 程 ID 必 正数：

```rust
pub fn sys_tkill(tid: i32, signo: u32) -> AxResult<isize> {
    if tid <= 0 {
        return Err(AxError::InvalidInput);
    }
    let sig = make_siginfo(signo, SI_TKILL)?;
    send_signal_to_thread(None, tid as Pid, sig)?;
    Ok(0)
}

pub fn sys_tgkill(tgid: i32, tid: i32, signo: u32) -> AxResult<isize> {
    if tgid <= 0 || tid <= 0 {
        return Err(AxError::InvalidInput);
    }
    let sig = make_siginfo(signo, SI_TKILL)?;
    send_signal_to_thread(Some(tgid as Pid), tid as Pid, sig)?;
    Ok(0)
}
```

代 4-7 tkill 和 tgkill 的参数

0 会被拒 ，避免 的 ID 类型 致非法内存 。

4.4

4.4.1

作 控制允 用 停正在 行的程序、将其放到后台 行或重新 起到前台。Shell 通信号和 程 作 控制：

- Ctrl-Z 向前台 程 送 SIGTSTP， 程收到后停止 行
- bg 命令向停止的 程 送 SIGCONT， 程在后台 行
- fg 命令将后台 程 拉到前台并 送 SIGCONT

waitpid Shell 据此更新作 列表。程 行后，父 程也 收到通知。

4.4.2

moss 使用 ProcessWaitStatus

```
pub enum ProcessWaitStatus {
    Stopped(Signo),
    Continued(Signo),
}

// 停止信号  理
SignalOSAction::Stop => {
    *thr.proc_data.wait_status.lock() =
Some(ProcessWaitStatus::Stopped(signo));
    wake_waiting_parent(&thr.proc_data);
    wait_for_continue(&thr.proc_data);
}
```

代 4-8 停止信号的 理流程

Stopped，再 醒正在等待的父 程。父 程从 wait 系 用返回， 取子 程的停止状 和 致停止的信号 号。

停止后的 程 入 wait_for_continue，在 函数中阻塞，直到收到 SIGCONT 或 SIGKILL：

```
fn wait_for_continue(proc_data: &ProcessData) {
    block_on(poll_fn(|cx| {
        let stopped = matches!(
            *proc_data.wait_status.lock(),
            Some(ProcessWaitStatus::Stopped(_))
        );
        if !stopped || proc_data.proc.is_zombie() {
            Poll::Ready(())
        } else {
            proc_data.stop_event.register(cx.waker());
            Poll::Pending
        }
    }));
}
```

代 4-9 等待 信号的

SIGKILL 和 SIGCONT 除外）。 保了停止的 程真正 停 行，而不是 理其他事件。

4.4.3

SIGCONT 使停止的 程恢 行。moss 在 送 信号 特殊 理：

```rust
pub fn send_signal_to_thread(tgid: Option<Pid>, tid: Pid, sig:
Option<SignalInfo>) -> AxResult<()> {
    let task = get_task(tid)?;
    let thread = task.try_as_thread().ok_or(AxError::OperationNotPermitted)?;

    if let Some(sig) = sig {
        let signo = sig.signo();
        if signo == Signo::SIGCONT {
            *thread.proc_data.wait_status.lock() =
Some(ProcessWaitStatus::Continued(signo));
            thread.proc_data.stop_event.wake();
            wake_waiting_parent(&thread.proc_data);
        } else if signo == Signo::SIGKILL {
            thread.proc_data.stop_event.wake();
        }
        // ...
        send_signal_thread_inner(&task, thread, sig);
    }
    Ok(())
}
```

代 4-10

Continued，再 醒 stop_event 上的等待者。停止的 程从 wait_for_continue 返回，重新参与 度。同 醒父 程，父 程可以通 waitpid

SIGKILL 也会 醒停止的 程，但不更新等待状 。 程从停止中恢 后直接 入退出流程，不再 行用 代 。

4.4.4

子 程停止或 后，父 程 当收到通知。moss 使用 child_exit_event 和 wait_status 配合 ：

```rust
fn wake_waiting_parent(proc_data: &ProcessData) {
    if let Some(parent) = proc_data.proc.parent()
        && let Ok(parent_data) = get_process_data(parent.pid())
    {
        parent_data.child_exit_event.wake();
    }
}
```

代 4-11

父 程 用 wait 或 waitpid child_exit_event 上等待。子 程状 化后 醒事件，父 程恢 行， 所有子 程的 wait_status，找到已 停止或 的子 程并返回相 状 。

moss 将停止与 状 接入 流程，使 Shell 能 正 管理作 状 。

4.5

4.5.1 ptrace

ptrace 是 Linux 的 程 接口，允 器 察和控制被 程。 器可以在被 程收到信号 停其 行， 寄存器和内存，决定是否将信号 被 程。

典型的 景包括：

- 被 程 非法地址，收到 SIGSEGV。 器截 信号， 示崩 位置和 用 ，而不是 程直接退出
- SIGTRAP。 器收到通知， 停 程并等待用 命令
- 用 向被 程 送 SIGINT。 器可以 将信号 程，也可以 弃信号并

moss 在 check_signals 的 增加了 ptrace

```
pub fn check_signals(
    thr: &Thread,
    uctx: &mut UserContext,
    restore_blocked: Option<SignalSet>,
) -> bool {
    crate::syscall::ptrace_wait_if_stopped(thr);

    let pid = thr.proc_data.proc.pid();
    if crate::syscall::ptrace_is_traced(pid) {
        let mask = !thr.signal.blocked();
        if let Some(sig) = thr.signal.dequeue_signal(&mask) {
            let signo = sig.signo();
            if crate::syscall::ptrace_intercept_signal(thr, signo) {
                return true;
            }
            do_exit(signo as i32, true);
        }
    }

    //
    // ...
}
```

代 4-12 信号 中的 ptrace

被 程在 信号前，先判断是否 于 ptrace 停止状 。如果 器尚未恢 程，程会在 ptrace_wait_if_stopped 中阻塞。 器通 PTRACE_CONT 或相 命令恢 程后，信号 。

接下来，内核判断 程是否正在被 。被 程收到未被屏蔽的信号 ， 用 ptrace_intercept_signal 通知 器。 器可以 信号、修改信号或决定是否 。如果 器 理了信号，函数返回 true，内核不再 理；如果 器 信号，函数返回 false，内核按正常流程 理信号。

4.5.2

ptrace moss

在 送信号 一条件：

```
if thread.signal.process().signal_ignored(signo)
    && crate::syscall::ptrace_mark_signal_stop(pid, signo)
{
    return Ok(());
}
send_signal_thread_inner(&task, thread, sig);
```

代 4-13 被忽略信号的 ptrace

即使信号被忽略，只要 程正在被 ，内核仍会 信号并触 ptrace 停止。 器可以 察到所有 送 被 程的信号，而不会因 程忽略信号而漏掉通知。

SIGHUP，但 器仍需要知道 程收到了 信号。

4.6

4.6.1 sys_pause

pause 系 用使 程 停 行，直到收到任意信号。moss 新增了 一系 用：

```
pub fn sys_pause(uctx: &mut UserContext) -> AxResult<isize> {
    let curr = current();
    let thr = curr.as_thread();

    // pause          -EINTR
    uctx.set_retval(-LinuxError::EINTR.code() as usize);

    block_on(poll_fn(|cx| {
        if check_signals(thr, uctx, None) {
            return Poll::Ready(());
        }
        let _ = curr.poll_interrupt(cx);
        Poll::Pending
    }));

    Err(AxError::Interrupted)
}
```

代 4-14 pause 系 用的

pause 在信号到 前一直阻塞，收到信号后 是返回 EINTR。它常用于等待子 程状 化或定 器到期。

4.6.2 sys_rt_sigtimedwait

原有 在 sigtimedwait 被信号 理函数中断 返回 0，但正 的返回 是 EINTR。moss 修正了 一 ：

```
let Ok(sig) = block_on(future::timeout(timeout, fut)) else {
    // 超
    signal.set_blocked(old_blocked);
    return Err(AxError::WouldBlock);
};
let Some(sig) = sig else {
    // 被信号  理函数中断
    return Err(AxError::Interrupted);
};
```

代 4-15 sigtimedwait 的返回 修正

等待超 返回 EAGAIN，被信号 理函数中断 返回 EINTR。 与 Linux 的 一致。

同 ，moss sigtimedwait 在恢 屏蔽集 的 机。无 是正常返回、超 是被中断，都 在返回前恢 原来的屏蔽集，避免屏蔽状 乱。

4.7

StarryOS 已 具 信号 送、接收、 理和用 理函数的基 框架，moss 没有重新 信号机制。本章的修改集中在信号生命周期的几个 ： 送前的权限 ，停止与 的状 管理， 器的信号 截，父 程的状 通知。

权限 防止非法信号 送，完善了 POSIX 安全 。停止与 机制支持作 控制，使 Shell 能 管理前后台作 。ptrace wait 系 用，子 程停止或 后父 程能 及 知状 化。

moss 的信号机制能 支持 Shell 作 控制、GDB POSIX 兼容的 程管理工具。